

Question 01:

Consider the following grammar as shown in the figure, which of the following sentences are in the language generated by this grammar

- a. babb
- b. bbbabb
- c. bbaaaaabc
- d. aaaaaa

$$\begin{aligned} \langle S \rangle &\rightarrow \langle A \rangle a \langle B \rangle b \\ \langle A \rangle &\rightarrow \langle A \rangle b \mid b \\ \langle B \rangle &\rightarrow b \end{aligned}$$

Part (a): babb

$\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b$
 $\langle S \rangle \rightarrow b a \langle B \rangle b$
 $\langle S \rangle \rightarrow b a b b$
 → Correct

Part (b): bbbabb

$\langle S \rangle \rightarrow \langle A \rangle a \langle B \rangle b$
 $\langle S \rangle \rightarrow \langle A \rangle b a \langle B \rangle b$
 $\langle S \rangle \rightarrow \langle A \rangle b b a \langle B \rangle b$
 $\langle S \rangle \rightarrow b b b a \langle B \rangle b$
 $\langle S \rangle \rightarrow b b b a b b$
 → Correct

Part (c): bbaaaaabc
 → Incorrect
 (presence of c)

Part (d): aaaaaa
 → Incorrect
 (absence of b)

Consider the following grammar as shown in the figure, which of the following sentences are in the language generated by this grammar

- a. abcd
- b. acccbd
- c. acccbcc
- d. accd
- e. ccc

$$\begin{aligned} \langle S \rangle &\rightarrow a \langle S \rangle c \langle B \rangle \mid \langle A \rangle \\ \langle A \rangle &\rightarrow c \langle A \rangle \mid c \\ \langle B \rangle &\rightarrow d \end{aligned}$$

Part (a): abcd

$\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle$
 $\langle S \rangle \rightarrow a \langle A \rangle c \langle B \rangle$
 $\langle S \rangle \rightarrow a c \langle A \rangle \mid c c \langle B \rangle$
 → Incorrect

Part (d): accd

$\langle S \rangle \rightarrow a \langle S \rangle c \langle B \rangle$
 $\langle S \rangle \rightarrow a \langle A \rangle c \langle B \rangle$
 $\langle S \rangle \rightarrow a c c \langle B \rangle$
 $\langle S \rangle \rightarrow a c c d$
 → Correct

Part (b): acccbd
 → Incorrect
 (presence of b)

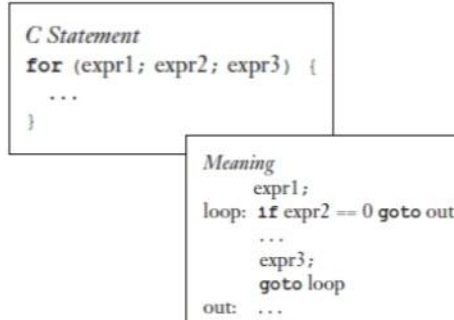
Part (e): ccc

$\langle S \rangle \rightarrow \langle A \rangle$
 $\langle S \rangle \rightarrow c \langle A \rangle$
 $\langle S \rangle \rightarrow c c \langle A \rangle$
 $\langle S \rangle \rightarrow c c c$
 → Correct

Part (c): acccbcc
 → Incorrect
 (presence of b)

Using the virtual machine instructions shown in the figures, give an operational semantic definition of the following:

- a. Java **do-while**
- b. Ada **for**
- c. C++ **if-then-else**
- d. C **switch**



a. (Java do-while) `do { ... }while(exp);`

Operational semantic definition =>

```

Loop: ...
    if !<rel_exp> goto out
    goto loop
Out: ...
  
```

b. (Ada for) `for I in min_val ... max_val loop`

Operational semantic definition =>

```

I = min_val
Loop: if I >= max_val goto out
    ...
    I = I + 1
    goto loop
Out: ...
  
```

c. (C++ if-else) `if(bool_exp) {stmts1;} else {stmts2;}`

Operational semantic definition =>

```

If <bool_exp> goto L1
goto L2
L1: stmts1;
L2: stmts2;
  
```

```
d. (C switch) switch(exp)
    {case <literal>: stmts1; break;
    default: stmts2; }
```

Operational semantic definition =>

```
If(exp_value == <literal>) goto L1
goto L2
L1: stmts1
L2: stmts2
```

Question 02:

Write a BNF description of the Boolean expressions of Java, including the operators '&&', '||', '!', and the relational expressions (==, <, <=, >, >=, !=).

<pre><assign> → <id> = <expr> <id> → A B C <expr> → <expr> + <term> <term> <term> → <term> * <factor> <factor> <factor> → (<expr>) <id></pre>

A == B ...
(A || B) == !(A && C)

```
<assign> → <id> = <bool_expr>
<id> → A | B | C | D | E
<bool_expr> → <bool_expr> || <bool_term>
             | <bool_term>
<bool_term> → <bool_term> && <bool_factor>
             | <bool_factor>
<bool_factor> → <id>
               | ! <bool_factor>
               | (<bool_expr>)
               | <rel_expr>
<rel_expr> → <bool_factor> == <bool_factor>
            | <bool_factor> != <bool_factor>
            | <bool_factor> < <bool_factor>
            | <bool_factor> <= <bool_factor>
            | <bool_factor> > <bool_factor>
            | <bool_factor> >= <bool_factor>
```

Question 03:

Using the below grammar, show a parse tree and a leftmost derivation for each of the following statements.

- $A = A * (B + (C * A))$
- $B = C * (A * C + A)$
- $A = A * (B + C)$

```

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
        | <id> * <expr>
        | ( <expr> )
        | <id>
    
```

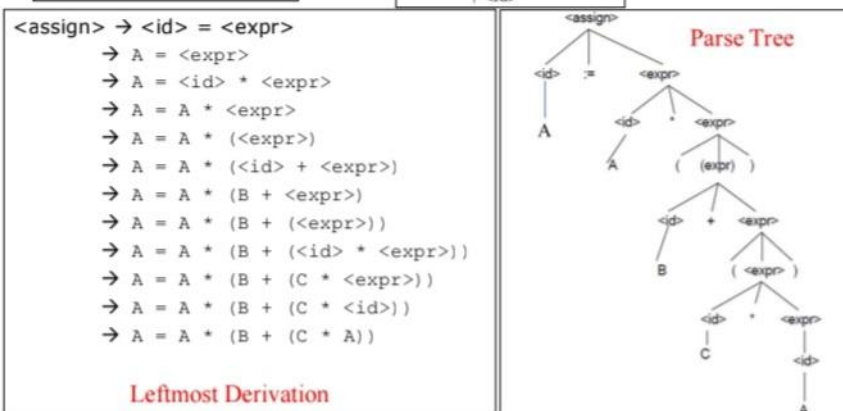
A = A * (B + (C * A))

Animation12

```

<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
        | <id> * <expr>
        | ( <expr> )
        | <id>
    
```

Given BNF Rules



Question 04:

Compute the weakest precondition for each of the following assignment statements and their postconditions:

- $a = 2 * (b - 1) - 1 \{a > 0\}$
- $b = (c + 10) / 3 \{b > 6\}$
- $a = a + 2 * b - 1 \{a > 1\}$
- $x = 2 * y + x - 1 \{x > 11\}$

Part (a):

```

a = 2 * (b - 1) - 1 {a > 0}
2 * (b - 1) - 1 > 0
2 * b - 3 > 0
b > 3 / 2
    
```

Part (c):

```

a = a + 2 * b - 1 {a > 1}
a + 2 * b - 1 > 1
2 * b > 2 - a
b > 1 - a / 2
    
```

Part (b):

```

b = (c + 10) / 3 {b > 6}
(c + 10) / 3 > 6
c + 10 > 18
c > 8
    
```

Part (d):

```

x = 2 * y + x - 1 {x > 11}
2 * y + x - 1 > 11
2 * y + x > 12
y > 6 - x / 2
    
```

Question 05:

Compute the weakest precondition for each of the following sequences of assignment statements and their postconditions:

a. $a = 2 * b + 1; b = a - 3 \quad \{b < 0\}$

b. $a = 3 * (2 * b + a); b = 2 * a - 1 \quad \{b > 5\}$

Part (a):

```
a = 2 * b + 1; (1)
b = a - 3 (2)
{b < 0}

(2) a - 3 < 0 → {a < 3}
(1) a = 2 * b + 1 {a < 3}
2 * b + 1 < 3
b < 1
```

Part (b):

```
a = 3 * (2 * b + a); (1)
b = 2 * a - 1 (2)
{b > 5}

(2) 2 * a - 1 > 5 → {a > 3}
(1) a = 3 * (2 * b + a) {a > 3}
3 * (2 * b + a) > 3
2 * b + a > 1
b > (1 - a) / 2
```

Question 06:

Compute the weakest precondition for each of the selection constructs and their postconditions:

```
a. if (a == b)
    b = 2 * a + 1
else
    b = 2 * a
{b > 1}
```

```
b. if (x < y)
    x = x + 1
else
    x = 3 * x
{x < 0}
```

```
c. if (x > y)
    y = 2 * x + 1
else
    y = 3 * x - 1
{y > 3}
```

if B then S1 else S2

$\frac{\{B \text{ and } P\}S1 \{Q\}, \{(not B) \text{ and } P\}S2 \{Q\}}{\{P\} \text{if } B \text{ then } S1 \text{ else } S2 \{Q\}}$

$\{P\} \text{if } B \text{ then } S1 \text{ else } S2 \{Q\}$

a. if (a == b) (1)

$b = 2 * a + 1 \quad (1) \quad b = 2 * a + 1 \{b > 1\}$

else $\rightarrow 2 * a + 1 > 1 \rightarrow a > 0$

$b = 2 * a \quad (2) \quad (2)$

$\{b > 1\} \quad b = 2 * a \{b > 1\}$

$\rightarrow 2 * a > 1 \rightarrow a > 1/2$

if B then S1 else S2

$\frac{\{B \text{ and } P\}S1 \{Q\}, \{(not B) \text{ and } P\}S2 \{Q\}}{\{P\} \text{if } B \text{ then } S1 \text{ else } S2 \{Q\}}$

$\{P\} \text{if } B \text{ then } S1 \text{ else } S2 \{Q\}$

Result: $a > 1/2$

b. if (x < y) (1)

x = x + 1 (1) x = x + 1 {x < 0}

else → x + 1 < 0 → x < -1

x = 3 * x (2) (2)

{x < 0} x = 3 * x {x < 0}

→ 3 * x < 0 → x < 0

if B then S1 else S2

$$\frac{\{B \text{ and } P\}S1 \{Q\}, \{(not B) \text{ and } P\}S2 \{Q\}}{\{P\}if B \text{ then } S1 \text{ else } S2 \{Q\}}$$

Result: x < -1

c. if (x > y) (1)

y = 2 * x + 1 (1) y = 2 * x + 1 {y > 3}

else → 2 * x + 1 > 3 → x > 1

y = 3 * x - 1 (2) (2)

{y > 3} x = 3 * x - 1 {y > 3}

→ 3 * x - 1 > 3 → x > 4/3

if B then S1 else S2

$$\frac{\{B \text{ and } P\}S1 \{Q\}, \{(not B) \text{ and } P\}S2 \{Q\}}{\{P\}if B \text{ then } S1 \text{ else } S2 \{Q\}}$$

Result: x > 4/3

Question 07:

Prove the following program is correct:

```

{n > 0}
count = n; (1)           {P} while B do S end {Q}
sum = 0; (2)             {I and B} S {I}
                           {I} while B do S end {I and (not B)}
while count <> 0 do
    sum = sum + count; (3)
    count = count - 1; (4)
end
{sum = 1 + 2 + . . . + n} (I)

```

(I)

$\{I\} \rightarrow \{ \text{sum} = (\text{count} + 1) + (\text{count} + 2) + \dots + n \}$

$\{I \text{ and } B\} \rightarrow \{ (\text{sum} = (\text{count} + 1) + (\text{count} + 2) + \dots + n) \\ \text{AND } (\text{count} \geq 0) \}$

(4)

$\{P1\} \text{ count} = \text{count} - 1 \{I \text{ and } B\}$

$\{P1\} \rightarrow \{ (\text{sum} = \text{count} + (\text{count} + 1) + (\text{count} + 2) + \dots + n) \\ \text{AND } (\text{count} \geq 1) \}$

$\{Q2\} = \{P1\}$

(3)

$\{P2\} \text{ sum} = \text{sum} + \text{count} \{Q2\}$

$\{P2\} \text{ sum} = \text{sum} + \text{count} \{ (\text{sum} = \text{count} + (\text{count} + 1) + \dots + n) \\ \text{AND } (\text{count} \geq 1) \}$

$\text{sum} + \text{count} = \text{count} + (\text{count} + 1) + \dots + n$

$\{P2\} \rightarrow \{ (\text{sum} = (\text{count} + 1) + (\text{count} + 2) + \dots + n) \\ \text{AND } (\text{count} \geq 1) \}$

$\{Q3\} = \{P2\}$

(2)

$\{P4\} \text{ sum} = 0 \{ (\text{sum} = (\text{count} + 1) + (\text{count} + 2) + \dots + n) \\ \text{AND } (\text{count} \geq 1) \}$

$\{P3\} \rightarrow \{ (0 = (\text{count} + 1) + (\text{count} + 2) + \dots + n) \\ \text{AND } (\text{count} \geq 1) \}$

$\{Q4\} = \{P3\}$

(1)

$\{P4\} \text{ count} = n \{Q4\}$

$\{P4\} \text{ count} = n \{ ((\text{count} + 1) + (\text{count} + 2) + \dots + n = 0) \\ \text{AND } (\text{count} \geq 1) \}$

$\{P4\} \rightarrow ((n + 1) + (n + 2) + \dots + n = 0) \text{ AND } (n \geq 1)$

$(n \geq 1) \implies (n > 0)$

The left operand of the AND operator in P4 is true for $0 = 0$

The precondition for all the statements in the program is:

$\{P\} \rightarrow \{n > 0\}$

Question 08:

Which of the following identifier forms is most readable? Support your decision

- a. `FirstName`
- b. `First_Name`
- c. `firstname`

`First_Name`

- Words in this identifier are properly spaced with an underscore “_”.
- It makes the identifier more readable for most people.
- It defines the exact meaning of the function and also does not confuse readers.

Question 09:

Consider the following JavaScript program.

Assume that the execution of this program is in the following order:

main calls sub1, then sub1 calls sub2, then sub2 calls sub3

a) Assuming static scoping, which declaration of x is the correct one for a reference of x?

- i. sub1
- ii. sub2
- iii. sub3

b) Repeat part a, but assume dynamic scoping

```
// the main program
var x;
function sub1() {
    var x;
    function sub2() {
        . . .
    }
}
function sub3() {
    . . .
}
```

Part (a):

- i. sub1
- ii. sub1
- iii. main

Part (b):

- i. sub1
- ii. sub1
- iii. sub1

Question 10:

Assume the following JavaScript program was interpreted:

- Under static-scoping rules, what value of x is displayed in function sub1?
- Under dynamic-scoping rules, what value of x is displayed in function sub1?

```
var x;  
function sub1() {  
    document.write("x = " + x + " ");  
}  
function sub2() {  
    var x;  
    x = 10;  
    sub1();  
}  
x = 5;  
sub2();
```

a. Static scoping:

x = 5

b. Dynamic scoping

x = 10

Question 11:

Consider the following JavaScript program.

List all the variables, along with the program units where they are declared, that are visible in the bodies of

- sub1
- sub2
- Sub3

Assuming static scoping is used.

```
var x, y, z;  
function sub1() {  
    var a, y, z;  
    function sub2() {  
        var a, b, z;  
        . . .  
    }  
    . . .  
}  
function sub3() {  
    var a, x, w;  
    . . .  
}
```

i. in sub1:

a sub1

y sub1

z sub1

x main

ii. in sub2:

a sub2

b sub2

z sub2

y sub1

x main

iii. in sub3:

a sub3

x sub3

w sub3

y main

z main

Question 12:

Consider the following C program.

For each of the four marked points in this function, list each visible variable, along with the number of the definition statement that defines it.

```
void fun(void) {  
    int a, b, c; //definition 1  
    . . .  
    while ( . . . ) {  
        int b, c, d; //definition 2  
        . . . < ----- 1  
        while ( . . . ) {  
            int c, d, e; //definition 3  
            . . . < ----- 2  
        }  
        . . . < ----- 3  
    }  
    . . . < ----- 4  
}
```

Point 1:

a	1
b	2
c	2
d	2

Point 2:

a	1
b	2
c	3
d	3
e	3

Point 3:

a	1
b	2
c	2
d	2

Point 4:

a	1
b	1
c	1

Question 13:

Consider the following skeletal C program.

```
void fun1(void); /* prototype */  
void fun2(void); /* prototype */  
void fun3(void); /* prototype */  
  
void main()      { int a, b, c; . . . }  
  
void fun1 (void) { int b, c, d; . . . }  
  
void fun2 (void) { int c, d, e; . . . }  
  
void fun3 (void) { int d, e, f; . . . }
```

Given the following calling sequences and assuming that dynamic scoping is used, what variables are visible during execution of the last function called in each of the following cases?

- a. main calls fun1; fun1 calls fun2; fun2 calls fun3.
- b. main calls fun1; fun1 calls fun3.
- c. main calls fun2; fun2 calls fun3; fun3 calls fun1.
- d. main calls fun3; fun3 calls fun1.
- e. main calls fun1; fun1 calls fun3; fun3 calls fun2.
- f. main calls fun3; fun3 calls fun2; fun2 calls fun1.

Part (a) d, e, f fun3 c fun2 b fun1 a main	Part (b) d, e, f fun3 b, c fun1 a main	Part (c) b, c, d fun1 e, f fun3 a main
Part (d) b, c, d fun1 e, f fun3 a main	Part (e) c, d, e fun2 f fun3 b fun1 a main	Part (f) b, c, d fun1 e fun2 f fun3 a main

Question 14:

Assume the following rules of associativity and precedence for expressions:

<i>Precedence</i>	<i>Highest</i>	* , / , not
		+ , - , & , mod
		- (unary)
		= , /= , < , <= , >= , >
		and
	<i>Lowest</i>	or , xor
<i>Associativity</i>	<i>Left to right</i>	

Show the order of evaluation of the following expressions by parenthesizing all sub-expressions and placing a superscript on the right parenthesis to indicate order.

For example, for the expression $a + b * c + d$, the order of evaluation would be represented as $((a + (b * c)^1)^2 + d)^3$

- i. $a * b - 1 + c$
- ii. $a * (b - 1) / c \text{ mod } d$
- iii. $(a - b) / c \ \& \ (d * e / a - 3)$
- iv. $-a \text{ or } c = d \text{ and } e$
- v. $a > b \text{ xor } c \text{ or } d \leq 17$
- vi. $-a + b$

- i. $(((a * b)^1 - 1)^2 + c)^3$
- ii. $(((a * (b - 1)^1)^2 / c)^3 \text{ mod } d)^4$
- iii. $(((a - b)^1 / c)^2 \ \& \ (((d * e)^3 / a)^4 - 3)^5)^6$
- iv. $(((-a)^1 \text{ or } (c = d)^2 \text{ and } e)^3)^4$
- v. $(((a > b)^1 \text{ xor } c)^2 \text{ or } (d \leq 17)^3)^4$
- vi. $(- (a + b)^1)^2$

Question 15:

Show the order of evaluation of the following expressions, assuming that there are no precedence rules and all operators associate right to left.

- i. $a * b - 1 + c$
- ii. $a * (b - 1) / c \bmod d$
- iii. $(a - b) / c \& (d * e / a - 3)$
- iv. $\neg a \text{ or } c = d \text{ and } e$
- v. $a > b \text{ xor } c \text{ or } d \leq 17$
- vi. $\neg a + b$

- i. $(a * (b - (1 + c)^1)^2)^3$
- ii. $(a * ((b - 1)^2 / (c \bmod d)^1)^3)^4$
- iii. $((a - b)^5 / (c \& (d * (e / (a - 3)^1)^2)^3)^4)^6$
- iv. $(\neg (a \text{ or } (c = (d \text{ and } e)^1)^2)^3)^4$
- v. $(a > (\text{xor } (c \text{ or } (d \leq 17)^1)^2)^3)^4$
- vi. $(\neg (a + b)^1)^2$

Question 16:

Let the function fun be defined as:

```
int fun (int *k) {  
    *k += 4;  
    return 3 * (*k) - 1;  
}
```

Suppose fun is used in a program as follows:

```
void main() {  
    int i = 10, j = 10, sum1, sum2;  
    sum1 = (i / 2) + fun(&i);  
    sum2 = fun(&j) + (j / 2);  
}
```

What are the values of sum1 and sum2:

- a. Operands in the expressions are evaluated left to right?
- b. Operands in the expressions are evaluated right to left?

a. (left to right) sum1 is 46; sum2 is 48

b. (right to left) sum1 is 48; sum2 is 46

Note that the new values of i and j in both cases is 14

Question 17:

Consider the following C program:

```
int fun (int *i) {  
    *i += 5;  
    return 4;  
}  
  
void main() {  
    int x = 3;  
    x = x + fun(&x);  
}
```

What are the values of x after the assignment statement in main, assuming:

- a. Operands in the expressions are evaluated left to right?
 - b. Operands in the expressions are evaluated right to left?
-
- a. (left to right) x is 7
 - b. (right to left) x is 12

Note that the new value of x in both cases is 8 before the addition.

Question 18:

Rewrite the following pseudo-code segment using a loop structure in the specified languages (all variables are integer):

a. C, C++, Java, or C#, b. Python, c. Ruby

```
k = j + 3  
loop:  
    if k > 10 then goto out  
    k = k + 1  
    i = 3 * k - 1  
    goto loop  
out: . . .
```

Which language, in this code, has:

- 1) the best writability,
- 2) the best readability,
- 3) and the best combination of the two.

Part (a) (C++)

```
for (k = j + 3; k <= 10; ) {  
    k++; i = 3 * k - 1; }
```

Part (b)

```
for k in range(j + 3, 11):  
    k = k + 1  
    i = 3 * k - 1
```

Part (c)

```
k = j + 3  
while k <= 10  
    k = k + 1  
    i = 3 * k - 1  
end
```

Question 19:

Rewrite the following code segment using a multiple selection statement in the specified languages (all variables are integer): a. C, C++, Java, or C#, b. Python, c. Ruby

```
if ((k == 1) || (k == 2)) j = 2 * k - 1  
if ((k == 3) || (k == 5)) j = 3 * k + 1  
if (k == 4) j = 4 * k - 1  
if ((k == 6) || (k == 7) || (k == 8)) j = k - 2
```

Which language, in this code, has the best writability, the best readability, and the best combination of the two.

C#:

```
switch (k)  
{  
  
    case 1: case 2: j = 2 * k - 1; break;  
    case 3: case 5: j = 3 * k + 1; break;  
  
    case 4: j = 4 * k - 1; break;  
    case 6: case 7: case 8: j = k - 2; break;  
  
    default: printf("error in switch, k = %d\n", k); break;  
}
```

```

j = 0
if k == 1 or k == 2 :
    j = 2 * k - 1
elif k == 3 or k == 5:
    j = 3 * k + 1
elif k == 4:
    j = 4 * k - 1
elif k == 6 or k == 7 or k == 8:
    j = k - 2
else:
    print ('error', j)
print (j)

j = case (k)
    when 1 , 2 then 2 * k - 1
    when 3 , 5 then 3 * k + 1
    when 4 then 4 * k - 1
    when 6..8 then k - 2
    else 0
end

if j == 0 then puts ("Error in k value.")
else puts (j)
end

```

Question 20:

Consider the following C program segment:

Rewrite the multiple selections statement using no gotos or breaks.

```

j = -3;
for (i = 0; i < 3; i++) {
    switch (j + 2) {
        case 3:
        case 2: j--; break;
        case 0: j += 2; break;
        default: j = 0;
    }
    if (j > 0) break;
    j = 3 - i;
}

```

```

j = -3;
for (i = 0; i < 3; i++){
    key = j + 2;
    if ((key == 3) || (key == 2)) j--;
    else if (key == 0) j += 2;
    else j = 0;

    if (j > 0) break;
    j = 3 - i;
}

```

Question 21:

Consider the following C program segment:

Rewrite it using if and goto statements in C.

```
j = -3;
for (i = 0; i < 3; i++) {
    switch (j + 2) {
        case 3:
        case 2: j--; break;
        case 0: j += 2; break;
        default: j = 0;
    }
    if (j > 0) break;
    j = 3 - i;
}
```

```
j = -3; i = 0;
LOOP: do {
    if ((key == 3) || (key == 2)) j--;
    else if (key == 0) j += 2;
    else j = 0;
    if (j > 0) goto OUT;
    j = 3 - i; i++;
    if(i == 3) goto OUT;
    goto LOOP;
}
OUT: . . .
```

Question 22:

Consider the following C program segment:

Rewrite it in Java without using a switch statement.

```
j = -3;
for (i = 0; i < 3; i++) {
    switch (j + 2) {
        case 3:
        case 2: j--; break;
        case 0: j += 2; break;
        default: j = 0;
    }
    if (j > 0) break;
    j = 3 - i;
}
```

```
j = -3;
for (i = 0; i < 3; i++) {
    if(j + 2 == 3 || j + 2 == 2) j--;
    else if(j + 2 == 0) j += 2;
    else j = 0;

    if (j > 0) break;
    j = 3 - i;
}
```